

Java Classes

The class is at the core of Java. It is the logical construct upon which the entire Java language is built because it defines the shape and nature of an object. It forms the basis for object-oriented programming in Java. Any concept you wish to implement in a Java program must be encapsulated within a class.

A class defines a new data type. Once defined, this new type can be used to create objects of that type. Thus, a class is a template for an object, and an object is an instance of a class. Because an object is an instance of a class, you will often see the two words object and instance used interchangeably.

When you define a class, you declare its exact form and nature by specifying the data that it contains and the code that operates on that data (or any one of it). The data, or variables, defined within a class are called instance variables. The code is contained within methods. Collectively, the methods and variables defined within a class are called members of the class. In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus, it is the methods that determine how a class' data can be used.

Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another. Java classes do not need to have a `main( )` method. You only specify one if that class is the starting point for your program. Further, some kinds of Java applications, such as applets, don't require a `main( )` method at all.

A class declaration only creates a template; it does not create an actual object.

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

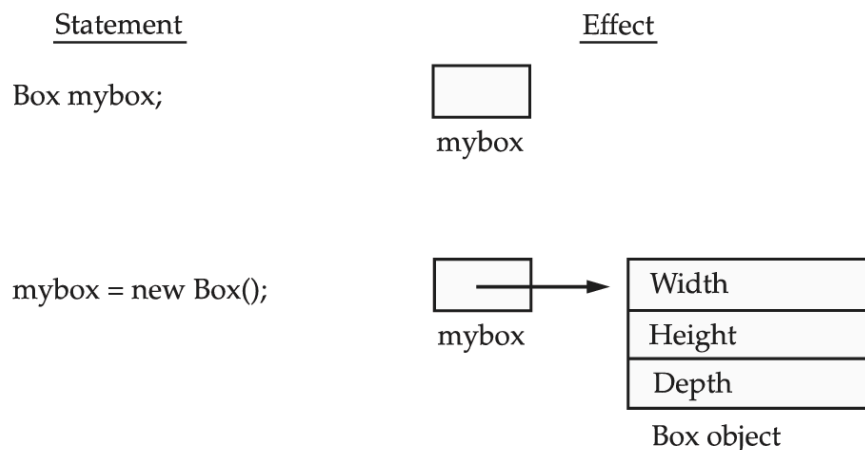
Box object will contain its own copies of the instance variables **width**, **height**, and **depth**. To access these variables, you will use the *dot* (.) operator. The dot operator links the name of the object with the name of an instance variable. For example, to assign the **width** variable of **mybox** the value 100, you would use the following statement:

```
mybox.width = 100;
```

You should call the file that contains this program `BoxDemo.java`, because the `main( )` method is in the class called `BoxDemo`, not the class called `Box`. First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the `new` operator. The `new` operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by `new`. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

```
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```

The first line declares **mybox** as a reference to an object of type **Box**. At this point, **mybox** does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to **mybox**. After the second line executes, you can use **mybox** as if it were a **Box** object. But in reality, **mybox** simply holds, in essence, the memory address of the actual **Box** object.



The class name followed by parentheses specifies the constructor for the class. A constructor defines what occurs when an object of a class is created. Most real-world classes explicitly define their own constructors within their class definition. However, if no explicit constructor is specified, then Java will automatically supply a default constructor. This is the case with Box.

Why you do not need to use new for things as integers or characters? The answer is that Java's primitive types are not implemented as objects. Rather, they are implemented as "normal" variables. Objects have many features and attributes that require Java to treat them differently than it treats the primitive types. By not applying the same overhead to the primitive types that applies to objects, Java can implement the primitive types more efficiently.

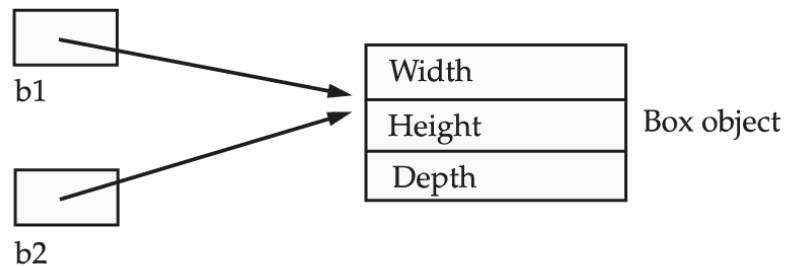
"new" allocates memory for an object during run time. The advantage of this approach is that your program can create as many or as few objects as it needs during the execution of your program. However, since memory is finite, it is possible that new will not be able to allocate memory for an object because insufficient memory exists. If this happens, a run-time exception will occur.

A class creates a logical framework that defines the relationship between its members. When you declare an object of a class, you are creating an instance of that class. Thus, a class is a logical construct. An object has physical reality. (That is, an object occupies space in memory.)

```
Box b1 = new Box();  
Box b2 = b1;
```

You might think that **b2** is being assigned a reference to a copy of the object referred to by **b1**. That is, you might think that **b1** and **b2** refer to separate and distinct objects. However, this would be wrong. Instead, after this fragment executes, **b1** and **b2** will both refer to the *same* object. The assignment of **b1** to **b2** did not allocate any memory or copy any part of the original object. It simply makes **b2** refer to the same object as does **b1**. Thus, any changes made to the object through **b2** will affect the object to which **b1** is referring, since they are the same object.

This situation is depicted here:



Although **b1** and **b2** both refer to the same object, they are not linked in any other way. For example, a subsequent assignment to **b1** will simply *unhook* **b1** from the original object without affecting the object or affecting **b2**. For example:

```
Box b1 = new Box();  
Box b2 = b1;  
// ...  
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

REMEMBER When you assign one object reference variable to another object reference variable, you are not creating a copy of the object, you are only making a copy of the reference.

```
Main.java  ::  Box.java  ::  
1 ~ public class Box {  
2     int a;  
3     int b;  
4 }
```

```
Main.java : Box.java :
1 public class Main
2 {
3     public static void main(String[] args) {
4         Box b1 = new Box();
5         Box b2 = b1;
6
7         b1.a=1;
8         b1.b=3;
9         System.out.println("Hello World " + b2.a + " " + b2.b);
10    }
11 }
```

input

Hello World 1 3

...Program finished with exit code 0
Press ENTER to exit console.

Basically b1 and b2 point to the same box object-

```
Main.java : Box.java :
1 public class Main
2 {
3     public static void main(String[] args) {
4         Box b1 = new Box();
5         Box b2 = b1;
6
7         b2.a=1;
8         b2.b=3;
9
10        System.out.println("Hello World " + b1.a + " " + b1.b);
11    }
12 }
```

input

Hello World 1 3

...Program finished with exit code 0
Press ENTER to exit console.

Unhooking b1 from b2-

```
Main.java  Box.java
1  public class Main
2  {
3      public static void main(String[] args) {
4          Box b1 = new Box();
5          Box b2 = b1;
6
7          b1.a=1;
8          b1.b=3;
9
10         b1 = null;
11         System.out.println("Hello World " + b2.a + " " + b2.b);
12     }
13 }
```

input

Hello World 1 3

...Program finished with exit code 0
Press ENTER to exit console.

? Is setting a reference to `null` the only way to “unhook” it from the object?

👉 Short answer: NO.

Setting to `null` is just **one way** to unhook a reference. You can also unhook it by reassigning it to another object.

◆ 1. What is really happening with references?

In **Java**, variables of class type **do NOT store objects directly**.

They store a **reference (address/pointer-like value)** to the object in memory.

So when you do:

```
Box b1 = new Box();
Box b2 = b1;
```

Memory looks like this:

```
b1 —> [ Box object ]
b2 —> ↑ same object
```

👉 Both refer to the **same object**

◆ 2. What does “unhooking” mean?

“Unhooking” simply means:

The reference variable stops pointing to that object.

There are **two main ways** to unhook:

✅ WAY 1: Assign **null**

```
b1 = null;
```

Now:

```
b1 → null
```

```
b2 → [ Box object ]
```

- ✓ Object still exists
 - ✓ b2 still refers to it
 - ✓ only b1 is unhooked
-

✅ WAY 2: Reassign to a new object

```
b1 = new Box();
```

Now:

```
b1 → [ new Box object ]
```

```
b2 → [ old Box object ]
```

👉 You didn't use **null**, but you **still unhooked b1** from the old object.

So yes — **reassignment also breaks the link.**

Key Insight

A reference variable can point to **only one object at a time**.

So whenever you do:

```
b1 = something_else;
```

Java will:

1. Disconnect **b1** from old object
2. Connect **b1** to new thing

◆ 3. What happens to the old object?

Now important concept (interview favorite 🧐):

If **no references point to an object**, it becomes:

 **eligible for garbage collection**

Example:

```
Box b1 = new Box();  
Box b2 = b1;
```

```
b1 = null;  
b2 = null;
```

Now:

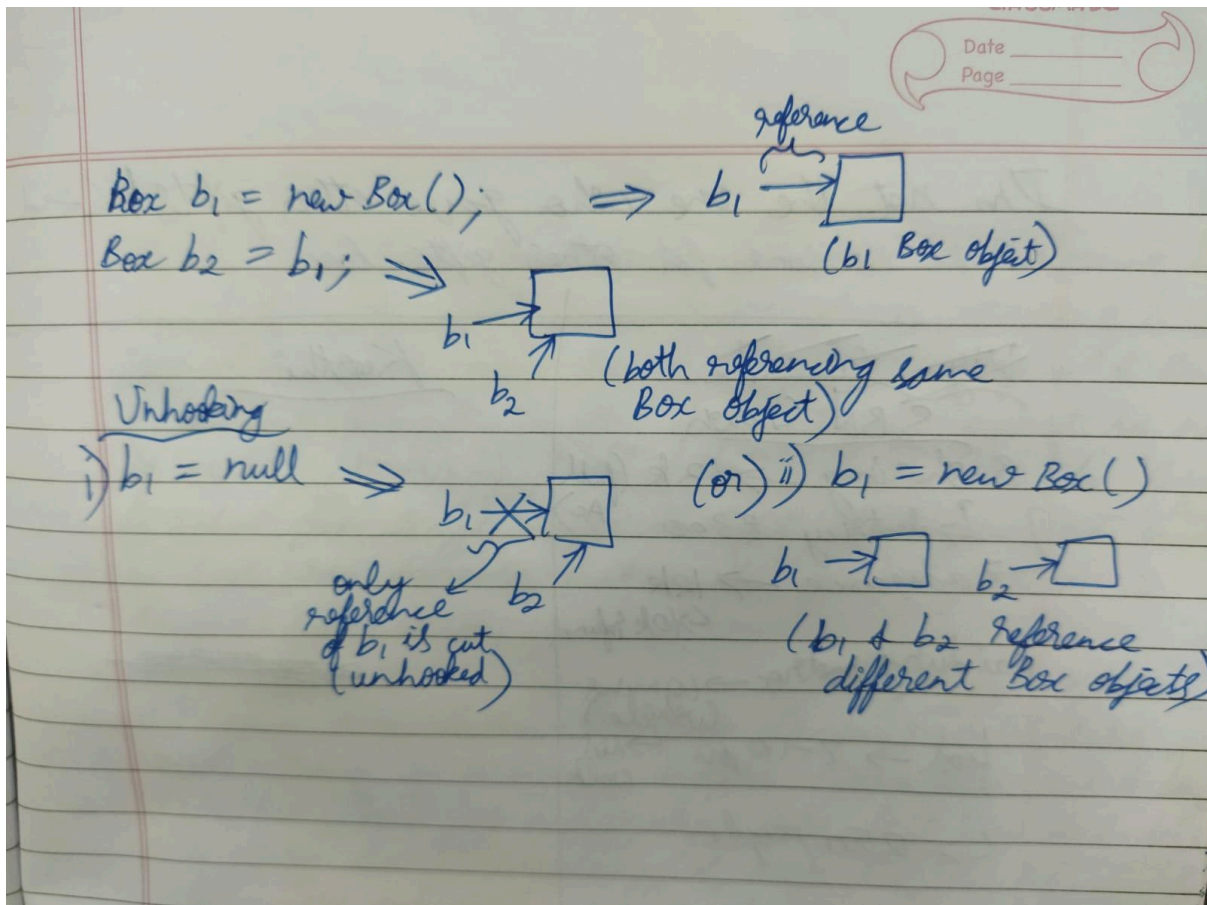
(no references) → [Box object]

👉 This object will be cleaned by Java's **Garbage Collector**.



Final Concept Summary

- Java variables store **references**, not actual objects
- `b2 = b1` → both point to **same object**
- Unhooking happens when:
 - you assign `null` OR
 - you assign a **new object**
- References are **independent variables**
- Object is deleted only when **no references exist**



This is the general form of a method:

```
type name(parameter-list) {  
    // body of method  
}
```

Here, *type* specifies the type of data returned by the method. This can be any valid type, including class types that you create. If the method does not return a value, its return type must be **void**. The name of the method is specified by *name*. This can be any legal identifier other than those already used by other items within the current scope. The *parameter-list* is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.

Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

```
return value;
```

Here, *value* is the value returned.

When an instance variable is accessed by code that is not part of the class in which that instance variable is defined, it must be done through an object, by use of the dot operator. However, when an instance variable is accessed by code that is part of the same class as the instance variable, that variable can be referred to directly.

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // display volume of a box  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}
```

Because the requirement for initialization is so common, Java allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a **constructor**. A constructor initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called when the object is created, before the “new” operator completes. Constructors look a little strange because they have no return type, not even void. This is because the implicit return type of a class’ constructor is the class type itself. It is the constructor’s job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

```

/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

```

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the **this** keyword. **this** can be used inside any method to refer to the *current* object. That is, **this** is always a reference to the object on which the method was invoked. You can use **this** anywhere a reference to an object of the current class' type is permitted.

To better understand what **this** refers to, consider the following version of **Box()**:

```

// A redundant use of this.
Box(double w, double h, double d) {
    this.width = w;
    this.height = h;
    this.depth = d;
}

```

This version of **Box()** operates exactly like the earlier version. The use of **this** is redundant, but perfectly correct. Inside **Box()**, **this** will always refer to the invoking object.

You can have local variables, including formal parameters to methods, overlap with the names of the class' instance variables. But then when a local variable has the same name as an instance variable, the local variable hides the instance variable. This is why width, height, and depth were not used as the names of the parameters to the **Box()** constructor inside the **Box** class (w, h, d are used instead). If they had been, then width, for example, would have referred to the formal parameter, hiding the instance variable width.

Because **this** lets you refer directly to the object, you can use it to resolve any namespace collisions that might occur between instance variables and local variables. For example, here is another version of **Box()**, which uses **width**, **height**, and **depth** for parameter names and then uses **this** to access the instance variables by the same name:

```
// Use this to resolve name-space collisions.
Box(double width, double height, double depth) {
    this.width = width;
    this.height = height;
    this.depth = depth;
}
```

It is explaining a concept called:

Variable shadowing (or hiding)

👉 When a **local variable or parameter** has the same name as an **instance variable**, the **local one “hides” the instance one** inside that method.

◆ First understand the two types of variables

Inside a class, you can have:

1. Instance variables (belong to object)

```
class Box {
    double width;
    double height;
    double depth;
}
```

These belong to the object.

2. Local variables / parameters (inside method)

```
Box(double width, double height, double depth) {
```

```
// these are parameters (local variables)

}
```

These exist only inside the constructor.

The Problem: SAME NAME

Now look carefully:

```
class Box {

    double width;

    double height;

    double depth;


    Box(double width, double height, double depth) {

        width = width;

        height = height;

        depth = depth;

    }

}
```

! This looks correct... but it is **WRONG**

Why is it wrong?

Because inside the constructor:

👉 `width` refers to the **parameter**, not the instance variable.

So this line:

```
width = width;
```

means:

```
parameter width = parameter width;
```

👉 It assigns the parameter to itself

👉 Instance variable is NEVER touched

So object values remain **0.0 (default)**

◆ This is what textbook means by “hiding”

The **local variable (parameter)** is **hiding** the **instance variable**

Because Java gives priority to the **nearest scope**

◆ How to fix this? → use **this**

Java gives a special keyword:

this

👉 **this** means: “**the current object**”

So now:

```
this.width
```

means:

👉 the instance variable of the object

✅ Correct version using **this**

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    Box(double width, double height, double depth) {  
        this.width = width;  
        this.height = height;  
        this.depth = depth;  
    }  
}
```

Now it means:

Left side

this.width → instance
variable

Right side

width →
parameter

So:

👉 object variable = parameter value ✅

◆ Why textbook said “we avoided using same names earlier”

Because beginners get confused with shadowing.

So they use:

```
Box(double w, double h, double d) {  
  
    width = w;  
  
    height = h;  
  
    depth = d;  
  
}
```

No confusion 👍

🔥 But professionals prefer same names + this

Because it's cleaner and standard practice:

```
this.width = width;
```

🎯 Key Concepts to remember

1. Java allows same name for:

- instance variable
- local variable / parameter

2. But local variable hides instance variable

3. Use `this` to access instance variable



One-line interview answer

If asked:

What is variable shadowing and how do you resolve it?

You say:

When a local variable or parameter has the same name as an instance variable, it hides the instance variable within that scope. This is called variable shadowing. It can be resolved using the `this` keyword to refer explicitly to the instance variable.



Final intuition

Think like this:

👉 `this.width` = “width of THIS object”

👉 `width` = “the local variable”

? What happens if you don't write any constructor in a class?

In **Java**, if you do not define any constructor at all, the compiler will **automatically create one for you**.

👉 This is called the **default constructor**

◆ What does the default constructor do?

The compiler-generated constructor:

```
Box() {  
    super(); // calls Object class constructor  
}
```

It does **nothing else**.

◆ What about instance variables?

All instance variables will get **default values** automatically:

Type	Default value
int	0
double	0.0
boolean	false
object reference	null
char	'\u0000'

◆ Example

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

No constructor written 🙅

Now:

```
Box b = new Box();  
System.out.println(b.width);
```

Output:

0.0

Because Java initialized it automatically.

🔥 Important rule

👉 The default constructor is created **ONLY IF** you don't write any constructor at all

! What if you write your own constructor?

```
class Box {
```

```
double width;
```

```
Box(double w) {  
    width = w;  
}  
}
```

Now Java **will NOT** create a default constructor

So this will fail ❌:

```
Box b = new Box(); // ❌ compile-time error
```

Because only constructor available is:

```
Box(double w)
```

◆ If you still want no-arg constructor?

You must **write it manually**:

```
class Box {  
    double width;  
  
    Box() { } // manual default constructor  
  
    Box(double w) {  
        width = w;  
    }  
}
```

}

◆ Why Java does this?

To ensure:

- objects are always properly initialized
 - constructor behavior is predictable
-



Summary

Situation	What happens
No constructor written	Java gives default constructor
At least one constructor written	Java does NOT give default
Want no-arg constructor + parameter constructor	You must define both



Interview one-liner

If asked:

What happens if a class has no constructor?

You answer:

If no constructor is defined, Java automatically provides a default no-argument constructor that initializes instance variables to their default values.



Quick mental model

- 👉 No constructor written → Java says: “I’ll create one for you”
- 👉 You write constructor → Java says: “Okay, you handle it yourself”

Let’s take your exact class:

```
class Box {  
  
    double width;  
  
    Box(double w) {  
        width = w;  
    }  
}
```

? Now what happens if you write:

```
Box b = new Box();
```

👉 **YES — your code will NOT compile at all ❌**

You will get a **compile-time error**.



Why does this happen?

Because in **Java**:

👉 The moment you define **any constructor**, Java **stops generating the default no-argument constructor**.

So your class has **ONLY ONE constructor**:

```
Box(double w)
```

But you are trying to call:

```
new Box() // no arguments
```

👉 That constructor **does not exist**

💥 Exact error you'll see

Something like:

```
constructor Box in class Box cannot be applied to given types;
```

```
required: double
```

```
found: no arguments
```

```
reason: actual and formal argument lists differ in length
```

◆ So will the program run?

✗ No

It won't even compile.

Because Java must **find a matching constructor at compile time**.

◆ How to fix it?

You have **two options**:

✓ Option 1: Pass the argument

```
Box b = new Box(10.5);
```

✓ Now it matches `Box(double w)`

✓ Option 2: Add a no-argument constructor yourself

```
class Box {  
    double width;  
  
    Box() { }    // 🙋 manually added  
  
    Box(double w) {  
        width = w;  
    }  
}
```

Now both are valid:

```
Box b1 = new Box();    // uses no-arg constructor
```

```
Box b2 = new Box(10.5); // uses parameter constructor
```



Key Rule (VERY IMPORTANT)

👉 Java only gives you a default constructor **if you don't write any constructor at all**

The moment you write even **one constructor**, you are responsible for **all constructor definitions**.

Garbage Collection

Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation. In some languages, such as C++, dynamically allocated objects must be manually released by use of a **delete** operator. Java takes a different approach; it handles deallocation for you automatically. The technique that accomplishes this is called *garbage collection*. It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. There is no explicit need to destroy objects as in C++. Garbage collection only occurs sporadically (if at all) during the execution of your program. It will not occur simply because one or more objects exist that are no longer used. Furthermore, different Java run-time implementations will take varying approaches to garbage collection, but for the most part, you should not have to think about it while writing your programs.

The finalize() Method

Sometimes an object will need to perform some action when it is destroyed. For example, if an object is holding some non-Java resource such as a file handle or character font, then you might want to make sure these resources are freed before an object is destroyed. To handle such situations, Java provides a mechanism called *finalization*. By using finalization, you can define specific actions that will occur when an object is just about to be reclaimed by the garbage collector.

To add a finalizer to a class, you simply define the **finalize()** method. The Java run time calls that method whenever it is about to recycle an object of that class. Inside the **finalize()** method, you will specify those actions that must be performed before an object is destroyed. The garbage collector runs periodically, checking for objects that are no longer referenced by any running state or indirectly through other referenced objects. Right before an asset is freed, the Java run time calls the **finalize()** method on the object.

The **finalize()** method has this general form:

```
protected void finalize( )
{
    // finalization code here
}
```

◆ 1. What does “out of scope” mean?

In **Java**, every variable has a **scope** (the region where it is valid).

Example:

```
void myMethod() {  
    Box b = new Box(); // b is created here  
} // scope of b ends here
```

👉 When execution leaves `myMethod()`:

- The variable `b` goes out of scope
- You **cannot access it anymore**

Important point

👉 The reference variable disappears,
! but the **object may still exist in memory**

Because Java uses **Garbage Collection (GC)**.

◆ 2. What happens when object goes “out of scope”?

When `b` goes out of scope:

```
Box b = new Box();
```

After the method ends:

`b` → ❌ (gone)

[Box object] → still in memory (temporarily)

👉 That object becomes **eligible for garbage collection**

◆ 3. What is `finalize()`?

`finalize()` is a method that Java **may call** before garbage collecting an object.

Example:

```
protected void finalize() {  
    System.out.println("Object destroyed");  
}
```

But here is the **critical part**:

Java does **NOT guarantee** when or even if `finalize()` will run.

! Why textbook says “don’t rely on `finalize()`”

Because:

- GC may run **later**
- or **much later**
- or **never** (before program ends)

So:

👉 You **cannot depend on `finalize()` to release resources**



Example of the problem

Suppose your object holds:

- file handle
- database connection

- network socket

If you rely on `finalize()` to close them ❌

👉 You may run out of resources before GC runs.

◆ 4. So how DO we handle resource cleanup?

👉 You should **manually release resources**

✓ Correct way (try-with-resources)

```
try(FileInputStream f = new FileInputStream("file.txt")) {  
    // use file  
} // automatically closed
```

✓ Or explicitly close

```
FileInputStream f = new FileInputStream("file.txt");  
f.close();
```

✓ Or create your own cleanup method

```
class Resource {  
    void close() {  
        // release resources  
    }  
}
```



Modern Java fact (important)

👉 `finalize()` is actually deprecated and removed in modern Java versions

Because it is **unreliable and unsafe**

◆ 5. Key difference: Out-of-scope vs Garbage Collection

Concept	Meaning
Out of scope	Variable is no longer accessible
Eligible for GC	Object has no references
Garbage collection	JVM may remove object from memory
<code>finalize()</code>	Might be called before GC (not guaranteed)



Simple mental model

Think like this:

- Scope ends → you **lose the handle**
- Object still floating in memory
- GC will clean it **whenever it feels like it**

Direct answer to your question

? What does “out of scope” mean and how do I deal with it?

👉 “Out of scope” means:

The reference variable is gone and cannot be used anymore.

👉 How to deal with it:

- Do **NOT** rely on `finalize()`
- Always **manually release resources**
- Use **try-with-resources** or **explicit close()**



Interview one-liner

If asked:

Why shouldn't we rely on `finalize()`?

You answer:

Because `finalize()` is not guaranteed to run immediately or at all. Resource cleanup should be done explicitly using `try-with-resources` or `close` methods.

// This class defines an integer stack that can hold 10 values

```
class Stack {  
  
    int stck[] = new int[10];  
  
    int tos;  
  
    // Initialize top-of-stack  
  
    Stack() {  
  
        tos = -1;
```

```

}

// Push an item onto the stack

void push(int item) {

    if(tos==9)

        System.out.println("Stack is full.");

    else

        stck[++tos] = item;

}

// Pop an item from the stack

int pop() {

    if(tos < 0) {

        System.out.println("Stack underflow.");

        return 0;

    }

    else

        return stck[tos--];

}

}

```

THIS CODE CAN BE MISUSED BECAUSE-

Excellent 🧠 this is actually a **very important Object-Oriented Programming concept** your textbook is trying to hint at.

The line:

“This leaves Stack open to misuse or mischief.”

means:

The internal data of the class is not protected.

Let's break it down clearly.

What is wrong with this Stack class?

Look at this line:

```
int stck[] = new int[10];
```

```
int tos;
```

Notice something?

👉 There is **no access modifier** (`private`, `public`, etc.)

In **Java**, if you don't specify an access modifier, it becomes:

default (package-private)

This means:

- Any class in the same package can directly access these variables.

Why is that dangerous?

Because someone can do this:

```
Stack s = new Stack();
```

```
s.tos = 100;    // ❌ break logic
```

```
s.stck[0] = 9999; // ❌ corrupt stack
```

Now your stack is:

- No longer safe
 - No longer following stack rules (LIFO)
 - Completely broken
-

That's what the book means by "misuse or mischief"

Users of the class can:

- Modify `tos`
- Modify `stck`
- Skip push/pop logic
- Break stack constraints

So your class is **not protecting its internal state**.

This violates a core OOP principle:

Encapsulation

Encapsulation means:

- 👉 Hide internal data
 - 👉 Expose only controlled methods
-

How to fix it properly

Make variables **private**

```
class Stack {  
  
    private int stck[] = new int[10];  
  
    private int tos;  
  
    Stack() {  
  
        tos = -1;  
  
    }  
}
```

```
void push(int item) {  
    if(tos == 9)  
        System.out.println("Stack is full.");  
    else  
        stck[++tos] = item;  
}
```

```
int pop() {  
    if(tos < 0) {  
        System.out.println("Stack underflow.");  
        return 0;  
    }  
    else  
        return stck[tos--];  
}  
}
```

Now:

```
Stack s = new Stack();
```

```
s.tos = 5;    // ❌ ERROR
```

```
s.stck[0] = 1; // ❌ ERROR
```

Perfect ✓

Now stack can only be used through `push()` and `pop()`.



Why is this important?

Because real-world classes may manage:

- Bank accounts
- Authentication tokens
- Database connections
- Financial transactions

If internal variables are exposed → huge security problems.
